

TUTORIAL Sprint 2

Para la realización del proyecto del Sprint 2, es imprescindible haber realizado el proyecto del Sprint anterior y visionar el video tutorial de este sprint. Este documento es un resumen de los comandos utilizados en dicho video tutorial.

Comandos HDFS

1. Abrir un terminal dentro de la máquina virtual y establecer el directorio de instalación de Hadoop como directorio actual
 - `cd /home/bigdata/hadoop-3.3.6/`
2. Arrancar HDFS
 - `sbin/start-dfs.sh`
3. Más adelante en el tutorial, cuando se hayan creado datos nuevos a través de Kafka Connect, se podrá listar el contenido de la carpeta que los contiene
 - `bin/hdfs dfs -ls /topics/fruits_topic/partition=0`

```
bigdata@bigdatapc:~/hadoop-3.3.6$ sbin/start-dfs.sh
Starting namenodes on [localhost]
Starting datanodes
Starting secondary namenodes [bigdatapc]
bigdata@bigdatapc:~/hadoop-3.3.6$ bin/hdfs dfs -ls /topics
Found 2 items
drwxr-xr-x - bigdata supergroup          0 2023-10-25 14:58 /topics/+tmp
drwxr-xr-x - bigdata supergroup          0 2023-10-25 14:58 /topics/fruits_topic
bigdata@bigdatapc:~/hadoop-3.3.6$ bin/hdfs dfs -ls /topics/fruits_topic
Found 1 items
drwxr-xr-x - bigdata supergroup          0 2023-10-25 14:58 /topics/fruits_topic/partition=0
bigdata@bigdatapc:~/hadoop-3.3.6$ bin/hdfs dfs -ls /topics/fruits_topic/partition=0
```

WebUI de HDFS

Cuando arrancamos HDFS, se arranca también una WebUI que nos aporta información general sobre el cluster de HDFS (Namenode, Secondary Namenode y Datanode) y también incluye una serie de utilidades donde destaca un navegador del sistema de ficheros de HDFS. Para utilizar esta funcionalidad se deben seguir los siguientes pasos:

1. Abrir el navegador de FireFox de la máquina virtual
2. Dirigirse a la url <http://localhost:9870>
3. Click en la pestaña Utilities que se encuentra a la derecha dentro de la franja verde de la parte superior
4. En el desplegable que aparece, click en Browse the file system

The screenshot shows the Hadoop WebUI interface. At the top, there's a navigation bar with tabs: Hadoop, Overview, Datanodes, Datanode Volume Failures, Snapshot, Startup Progress, and Utilities. The Utilities tab is selected, and a dropdown menu is open, showing options: Browse the file system, Logs, Log Level, Metrics, Configuration, Process Thread Dump, and Network Topology. The main content area shows the Overview for 'localhost:9000' (active). Below this, there's a table with system information.

Property	Value
Started:	Tue Oct 24 15:02:21 +0200 2023
Version:	3.3.6, r1be78238728da9266a4f88195058f08fd012bf9c
Compiled:	Sun Jun 18 10:22:00 +0200 2023 by ubuntu from (HEAD detached at release-3.3.6-RC1)
Cluster ID:	CID-3ab5777d-43ce-4df6-b71a-5604a3260017
Block Pool ID:	BP-495285225-127.0.1.1-1697135071914

Comandos de la plataforma Confluent (Ecosistema Kafka)

La plataforma Confluent es una plataforma basada en Kafka y que tiene 2 versiones: una comercial y otra Open-Source (esta última es la que está instalada en la máquina virtual y es la que utilizaremos para los Sprints). A continuación se detallan los comandos utilizados en el vídeo tutorial de este Sprint:

1. Dirigirse a la carpeta de instalación de la plataforma Confluent
 - `cd /home/bigdata/confluent-7.5.1/`
2. Arrancar Kafka-Connect y todos los servicios requeridos para ello (Zookeeper, Kafka, Schema Registry)
 - `confluent local services connect start`
3. Antes de cargar el conector (sink) de Parquet (cliente HDFS) en Kafka-Connect, observar el contenido de su fichero de configuración
 - `cat /home/bigdata/Descargas/hdfs3-parquet-fruits.json`
4. Cargar dicho conector en Kafka-Connect
 - `bin/confluent local services connect connector load hdfs3-parquet-fruits --config /home/bigdata/Descargas/hdfs3-parquet-fruits.json`
5. Comprobar que el conector se ha cargado correctamente en Confluent
 - `bin/confluent local services connect connector status hdfs3-parquet-fruits`
6. Arrancar un consumidor de Kafka en formato Avro del topic `fruits_topic` para comprobar que los datos se introducen correctamente en este topic
 - `bin/kafka-avro-console-consumer --bootstrap-server localhost:9092 --topic fruits_topic`
 - Más adelante, cuando se haya ya hecho uso del conector para introducir datos en HDFS, este consumidor se puede detener situándose en su terminal y realizando la combinación de teclas `Control+C`
7. Observar las primeras líneas del fichero de datos que vamos a volcar a dicho topic. Los datos están en formato JSON.
 - `head /home/bigdata/Descargas/fruits.json`
8. Volcar dichos datos utilizando un productor de Kafka en formato Avro. Para ello, utilizaremos la redirección de contenido del terminal de Ubuntu (símbolo `|`) y aportaremos el esquema de los datos (nombres de columnas y tipos) añadiendo la propiedad `value.schema`
 - `cat /home/bigdata/Descargas/fruits.json | bin/kafka-avro-console-producer --broker-list localhost:9092 --topic fruits_topic --property value.schema='{ "type": "record", "name": "fruits", "fields": [{ "name": "id", "type": "int" }, { "name": "name", "type": "string" }, { "name": "rating", "type": "double" }] }'`

```

bigdata@bigdatapc:~$ cd /home/bigdata/confluent-7.5.1/
bigdata@bigdatapc:~/confluent-7.5.1$ confluent local services connect start
The local commands are intended for a single-node development environment only, NOT for production usage. See more: https://docs.confluent.io/current/cli/index.html
As of Confluent Platform 8.0, Java 8 will no longer be supported.

Using CONFLUENT_CURRENT: /tmp/confluent.016675
Starting ZooKeeper
ZooKeeper is [UP]
Starting Kafka
Kafka is [UP]
Starting Schema Registry
Schema Registry is [UP]
Starting Connect
Connect is [UP]
bigdata@bigdatapc:~/confluent-7.5.1$ cat /home/bigdata/Descargas/hdfs3-parquet-fruits.json
{
  "name": "hdfs3-parquet-fruits",
  "config": {
    "connector.class": "io.confluent.connect.hdfs3.Hdfs3SinkConnector",
    "tasks.max": "1",
    "topics": "fruits_topic",
    "hdfs.url": "hdfs://localhost:9000",
    "flush.size": "5",
    "key.converter": "org.apache.kafka.connect.storage.StringConverter",
    "value.converter": "io.confluent.connect.avro.AvroConverter",
    "value.converter.schema.registry.url": "http://localhost:8081",
    "confluent.topic.bootstrap.servers": "localhost:9092",
    "confluent.topic.replication.factor": "1",
    "format.class": "io.confluent.connect.hdfs3.parquet.ParquetFormat"
  }
}
bigdata@bigdatapc:~/confluent-7.5.1$ █

```

```

bigdata@bigdatapc:~/confluent-7.5.1$ bin/confluent local services connect connector load hdfs3-parquet-fruits --config /home/bigdata/Descargas/hdfs3-parquet-fruits.json
The local commands are intended for a single-node development environment only, NOT for production usage. See more: https://docs.confluent.io/current/cli/index.html
As of Confluent Platform 8.0, Java 8 will no longer be supported.

{
  "name": "hdfs3-parquet-fruits",
  "config": {
    "connector.class": "io.confluent.connect.hdfs3.Hdfs3SinkConnector",
    "tasks.max": "1",
    "topics": "fruits_topic",
    "hdfs.url": "hdfs://localhost:9000",
    "flush.size": "5",
    "key.converter": "org.apache.kafka.connect.storage.StringConverter",
    "value.converter": "io.confluent.connect.avro.AvroConverter",
  }
}

```

```

bigdata@bigdatapc:~/confluent-7.5.1$ bin/confluent local services connect connector status hdfs3-parquet-fruits
The local commands are intended for a single-node development environment only, NOT for production usage. See more: https://docs.confluent.io/current/cli/index.html
As of Confluent Platform 8.0, Java 8 will no longer be supported.

{
  "name": "hdfs3-parquet-fruits",
  "connector": {
    "state": "RUNNING",
    "worker_id": "127.0.1.1:8083"
  },
  "tasks": [
    {
      "id": 0,
      "state": "RUNNING",
    }
  ]
}

```

```

bigdata@bigdatapc:~/confluent-7.5.1$ bin/kafka-avro-console-consumer --bootstrap-server localhost:9092 --topic fruits_topic
[2023-10-25 14:54:49,186] INFO KafkaAvroDeserializerConfig values:
  auto.register.schemas = true
  avro.reflection.allow.null = false
  avro.use.logical.type.converters = false
  basic.auth.credentials.source = URL
  basic.auth.user.info = [hidden]
  bearer.auth.cache.expiry.buffer.seconds = 300
  bearer.auth.client.id = null
  bearer.auth.client.secret = null

```

```

bigdata@bigdatapc:~/confluent-7.5.1$ head /home/bigdata/Descargas/fruits.json
{"id":1,"name":"Banana","rating":9.3}
{"id":2,"name":"Orange","rating":9.0}
{"id":3,"name":"Kiwi","rating":7.8}
{"id":4,"name":"Apple","rating":8.5}
{"id":5,"name":"Grape","rating":7.9}
{"id":6,"name":"Watermelon","rating":8.8}
{"id":7,"name":"Melon","rating":7.7}
{"id":8,"name":"Mango","rating":6.6}
{"id":9,"name":"Lemon","rating":9.6}
{"id":10,"name":"Cherry","rating":8.2}
bigdata@bigdatapc:~/confluent-7.5.1$ cat /home/bigdata/Descargas/fruits.json | bin/kafka-avro-console-producer --broker-list localhost:9092 --topic fruits_topic --property value.schema='{\"type\":\"record\",\"name\":\"fruits\",\"fields\":[{\"name\":\"id\",\"type\":\"int\"},{\"name\":\"name\",\"type\":\"string\"},{\"name\":\"rating\",\"type\":\"double\"}]}'

```

Shell de Hive

Una vez que hemos introducido datos en HDFS, vamos a utilizar Hive para realizar consultas analíticas, para ello, vamos a utilizar beeline que es una consola que proporciona Hive para poder lanzar consultas HiveQL:

1. Abrir un terminal nuevo y dirigirse a la carpeta de instalación de Hive
 - `cd /home/bigdata/apache-hive-3.1.3-bin/`
2. Arrancar el terminal beeline
 - `bin/beeline -u jdbc:hive2://`
3. Una vez dentro del terminal, crear una tabla con los datos introducidos en HDFS. Se debe aportar el esquema de la tabla (nombre y tipo de las columnas en orden)
 - `CREATE EXTERNAL TABLE fruits(id int, name string, rating double) STORED AS parquet LOCATION '/topics/fruits_topic/partition=0';`
4. Lanzar una consulta analítica para comprobar que se resuelve correctamente
 - `SELECT avg(rating) AS avg_rating FROM fruits;`
5. Salir de beeline
 - `!q`

```
bigdata@bigdatapc:~/hadoop-3.3.6$ cd /home/bigdata/apache-hive-3.1.3-bin/
bigdata@bigdatapc:~/apache-hive-3.1.3-bin$ bin/beeline -u jdbc:hive2://
```

```
Connected to: Apache Hive (version 3.1.3)
Driver: Hive JDBC (version 3.1.3)
Transaction isolation: TRANSACTION_REPEATABLE_READ
Beeline version 3.1.3 by Apache Hive
0: jdbc:hive2://> CREATE EXTERNAL TABLE fruits(id int, name string, rating double) STORED AS parquet LOCATION '/topics/fruits_topic/partition=0';
OK
No rows affected (1.118 seconds)
0: jdbc:hive2://> SELECT avg(rating) AS avg_rating FROM fruits;|
```

Detener servicios de Confluent

Una vez que ya hemos comprobado que los datos se han cargado correctamente en HDFS y que hemos creado una tabla en Hive con esos datos, ya se puede detener los servicios de Confluent (Kafka-Connect, Schema Registry, Kafka, Zookeeper):

- `confluent local services stop`

```
bigdata@bigdatapc:~/confluent-7.5.1$ confluent local services stop
The local commands are intended for a single-node development environment only, NOT for production usage. See more: https://docs.confluent.io/current/cli/index.html
As of Confluent Platform 8.0, Java 8 will no longer be supported.

Using CONFLUENT_CURRENT: /tmp/confluent.016675
Control Center is [DOWN]
ksqlDB Server is [DOWN]
Stopping Connect
Connect is [DOWN]
Kafka REST is [DOWN]
Stopping Schema Registry
Schema Registry is [DOWN]
Stopping Kafka
```

Detener HDFS

Una vez terminado el ejercicio, es imprescindible parar el servicio de HDFS adecuadamente para evitar que se puedan corromper el sistema de ficheros de HDFS:

1. Desde un terminal, dirigirse a la carpeta de instalación de Hadoop
 - `cd /home/bigdata/hadoop-3.3.6/`
2. Ejecutar el script que detiene Namenode, Secondary namenode y Datanode
 - `sbin/stop-dfs.sh`

```
bigdata@bigdatapc:~/hadoop-3.3.6$ sbin/stop-dfs.sh
Stopping namenodes on [localhost]
Stopping datanodes
Stopping secondary namenodes [bigdatapc]
bigdata@bigdatapc:~/hadoop-3.3.6$
```

Salir de la máquina virtual

Una vez que se ha detenido HDFS, ya podemos apagar la máquina virtual:

1. Dentro de la máquina virtual, click en el símbolo de la energía en la parte superior del estado de Ubuntu
2. Click en la opción Apagar / cerrar sesión
3. Click en Apagar...

